

Candidates Feature: Real Data Schema and Design Document

Project: HR Dashboard, Candidates Module **Status:** Schema finalized, ready for implementation **Date:** 2026-08-27

1. Purpose

The HR Dashboard's Candidates section currently shows **fake sample data** that's hardcoded directly into the app (in `main/frontend/src/data/hrMockData.js`), rather than pulling from a real database. This document lays out the finalized data structure (called a "schema," basically the blueprint for what information each candidate record holds) and the design decisions needed to turn this into a **real, database-backed feature** (using Firebase, the cloud database this project runs on) that is:

- **Fast.** Minimal delay reading data, and no unnecessary back-and-forth trips to the database.
- **Fresh.** The data shown reflects what's actually true right now, or close to it.
- **Careful with usage limits.** Firebase charges (or limits) based on how many times data is read, so this design avoids patterns already identified as wasteful elsewhere in this project: things like listening to an entire collection with no filter, loading everything at once instead of in pages, making one extra request per item in a list instead of one request for the whole list, and reading a record back immediately after writing to it.

Every decision below was weighed from two angles at once: what an experienced HR professional would actually need day to day, and what keeps Firebase's costs and performance reasonable.

2. Background: What Exists Today

The backend code (`main/backend/controllers/hrDeskController.js`) already has a `candidates` collection (a "collection" in Firestore, the database this app uses, is like a folder of records) wired up through a generic, reusable set of create/read/update/delete functions, the same set already used for employee records. The frontend's fake sample data file (`hrMockData.js`) defines the shape that this collection was originally modeled on, along with two related sample lists: interviews and feedback entries.

`HrDeskContext` (`main/frontend/src/context/HrDeskContext.jsx`) is the shared place in the frontend code that already fetches employee, candidate, interview, and attendance data all at once, in

a single batch, and only for people with the HR or founder role. It then makes that data available to every page that needs it, so individual pages don't each run their own separate fetch. Any real candidates implementation should plug into this existing setup rather than building something new.

3. Candidate Record Layout (Finalized)

FIELD	TYPE	WHAT IT HOLDS
id	Automatically generated by Firestore	A unique ID for the record
name	Text	The candidate's full name
email	Text	Contact email address
phone	Text	Contact phone number
location	Text	The candidate's current city or location
skills	A list of text tags	Skill tags, for example <code>['React', 'Node.js']</code>
experience	Number	Years of experience. Changed from loose free text like "4 yrs" to a plain number, so candidates can actually be filtered or sorted by it, for example "show me candidates with 3 to 5 years of experience"
education	Text	Highest qualification or institute attended
currentCTC	Number (new field)	The candidate's current salary
expectedSalary	Number	Expected salary. Changed from loose free text like "₹18 LPA" to a number, for the same filtering and sorting reason as experience
currentCompany	Text	Current employer
noticePeriod	Text or number (new field)	Notice period required at the candidate's current job. The exact format, days versus free text, will be finalized during implementation
portfolio	Text	A link to the candidate's portfolio or personal website (can be left blank)
source	A fixed set of options	Where the application came from (see section 4)
stage	A fixed set of options	Where the candidate currently is in the hiring pipeline (see section 4)
appliedFor	Text	The job title or position applied for
appliedOn	A proper date/time value	The application date. Changed from a plain text date like "2026-08-27" to a real timestamp, so records sort correctly and pages of results load properly
resumeFileName	Text	The original resume file's name (see section 5)
resumeUrl	Text (new field)	A link to download the resume from Firebase's file storage (see section 5)
rejectionReason	A fixed set of options (new field)	Why the candidate was rejected, or why they turned down an offer (see section 6)
assignedRecruiter	Text (new field)	The HR person responsible for this candidate

FIELD	TYPE	WHAT IT HOLDS
lastUpdatedBy	Text (new field)	A record of which user made the most recent change, for accountability
nextInterview	An object holding a few related values (new field)	A quick summary of the candidate's next scheduled interview: its date, type, and interviewer (see section 7)
created_at	A date/time value, set automatically	Recorded the moment the record is created, matching how the rest of the backend already works
updated_at	A date/time value, set automatically	Recorded every time the record is edited, matching how the rest of the backend already works

Why these changes from the original sample data?

- `experience` and `expectedSalary` used to be loose text like `"4 yrs"` or `"₹18 LPA"`. That makes it impossible to filter or run reports (like calculating an average expected salary, or filtering by an experience range) without unreliable text-parsing tricks. Turning them into plain numbers fixes this at the source.
- `appliedOn` used to be a plain text date. A proper date/time value lets the database sort records chronologically and search date ranges (like "candidates who applied in the last 30 days") directly, which is both faster and cheaper than pulling everything and sorting it after the fact in the browser.

4. Stage and Source Options (Finalized)

4.1 Candidate Stage

Where a candidate currently stands in the hiring pipeline:

1. Applied
2. Screening
3. HR Round
4. Technical Round
5. Final Interview
6. Offer Sent
7. Joined
8. Offer Declined (*new*)
9. Rejected
10. On Hold (*new; this one isn't a step in sequence, it's a pause that can apply at any point in the pipeline, not a final outcome*)

Why these two were added:

- **Offer Declined.** Previously, a candidate turning down an offer looked identical to the company rejecting the candidate; both were lumped under "Rejected." Splitting them apart lets HR track

"how many people who got offers turned them down" as its own separate number, which is a meaningfully different signal than the company saying no.

- **On Hold.** This covers situations where a candidate's process is paused, for example the role gets frozen or the budget is put on hold, rather than closed for good. Without this option, HR previously had no way to mark someone as "paused" without incorrectly marking them as rejected.

4.2 Application Source

Where the candidate's application came from:

1. LinkedIn
2. Naukri
3. Indeed
4. Monster
5. Internal Portal
6. Referral
7. Manual Upload
8. Company Website / Career Page (*new*)

Why this was added: this anticipates a future public application form on the company's own careers page. Without this option, applications coming in through that page would have no accurate source to record.

5. Resume Storage (Finalized)

Approach: store both a filename and a storage link.

- When a resume is uploaded, it's saved into Firebase's file storage, and a download link is captured **once, at the moment of upload**, then saved onto the candidate's record as `resumeUrl`, alongside `resumeFileName`.
- **Why not generate the link fresh each time it's needed instead?** HR often opens a candidate's resume multiple times while screening them. Generating a fresh download link every single time would mean an extra request to Firebase's storage system per view. Saving the link once, at upload time, turns a cost that repeats every time into a cost that happens only once, which directly supports the goal of staying within usage limits.

Additional rules:

- **File type:** PDF only. Word documents and other formats aren't accepted.
 - **File naming:** the original filename plus an added serial number, to prevent name clashes in storage when different candidates (or the same candidate re-uploading) use similar file names.
 - **Replacing a resume:** when a candidate uploads a new resume, the old file is deleted from storage. No version history is kept; only the current resume ever exists at any given time.
-

6. Rejection Reason (Finalized)

Approach: a dropdown with fixed options, not a free-text box.

Using a fixed, structured list of reasons lets HR run meaningful reports later, such as "what's the single biggest reason we're losing candidates at this stage," without needing to make sense of inconsistent free-text entries.

Proposed options (exact final wording may be refined during implementation):

- Salary Mismatch
- Skill Gap
- Culture Fit
- Position Closed/On Hold
- Candidate Withdrew
- Better Candidate Selected
- Notice Period Too Long
- Other

This applies to both the "Rejected" stage and the "Offer Declined" stage. Both outcomes benefit from a structured reason, since "why do candidates turn down our offers" is just as valuable a report as "why do we reject candidates."

7. Interview and Feedback Data (Finalized, a Hybrid Approach)

Interviews and feedback have their own needs that go beyond just being "attached to a candidate." For example, an interviewer needs to see their own schedule across *every* candidate they're interviewing, and HR may want a "today's interviews" view spanning the whole pipeline. That ruled out simply storing a list of interviews inside each candidate's own record, because Firestore, the database this app uses, can't efficiently search across many candidates' nested lists at once for views like that.

Decision: a hybrid structure.

1. **The full interview and feedback details** stay in their own separate collections, called `interviews` and `feedbackEntries`, each one linked back to a candidate through a `candidateId` field. This keeps the ability to search across all candidates at once, for example "every interview scheduled today," or "every piece of feedback a specific interviewer has given."
2. **The candidate's own record additionally carries a small summary field**, `nextInterview`, holding just the date, type, and interviewer for that candidate's next upcoming interview. This field gets updated whenever an interview is scheduled or rescheduled for that person.
3. **Full interview history and past feedback are only loaded when someone opens a candidate's individual detail page**, not when browsing the main candidates list.

Why this matters for performance: without that small summary field on each candidate's own record, showing "next interview date" in the candidates list would require one extra database request *per candidate shown*. That's a classic scaling problem where the number of requests grows right along

with the number of candidates on screen. The summary field avoids this entirely: the date is already sitting right there on the record being read anyway, at the cost of keeping a small bit of duplicated information in sync, which is easy to manage since it only changes when an interview is scheduled or rescheduled, an infrequent event.

8. Ownership and Accountability Fields (Finalized)

Two separate fields, serving two separate purposes:

- **assignedRecruiter**. The HR person responsible for, and driving forward, this candidate. Set by hand when a candidate is added to the system, or when they're reassigned to a different recruiter. This answers the question "who owns moving this candidate forward?"
- **lastUpdatedBy**. An accountability record, set automatically every time the record is edited, alongside the existing `updated_at` timestamp. This answers a different question: "who last touched this record?" This matters when multiple HR people are working the same pipeline.

Both fields are kept rather than picking just one, because ownership and an accountability trail answer two different questions, and neither one can stand in for the other.

9. Performance and Usage-Limit Considerations

This design follows the project's broader plan for staying within Firebase's usage limits. The specific practices that apply to the Candidates feature:

- **One shared data source, not several.** Candidates should be fetched once through the existing `HRDeskContext`, not separately by every individual component that happens to need candidate data.
- **Restricted by role.** Only people with the HR or founder role should ever trigger a candidates fetch at all; no other role should be watching this collection for changes.
- **Loaded in pages, not all at once.** The candidates list should load in batches (for example, 20 to 25 at a time) rather than pulling the entire collection in one go, with a sensible default filter (like showing only active, open candidates) so old, closed-out records aren't loaded unless someone specifically asks for them.
- **Filtering happens in the database, not after the fact.** Filters for status, position, or source should be applied as part of the database query itself, not fetched in bulk and then filtered afterward in the browser.
- **Live updates only where freshness genuinely matters.** Only views where up-to-the-second accuracy really counts, like a live pipeline board, should use a real-time connection that pushes updates automatically. Reporting and export views should just read the data once when opened.
- **No reading data back right after writing it.** After updating a candidate's stage or details, the screen should update using the change that was just made locally, rather than immediately re-fetching the same record from the database to confirm it.

- **Store a summary instead of making extra requests per item.** As explained in section 7, summary information needed for list views, like `nextInterview`, is stored directly on the candidate's own record instead of requiring an extra request per row to piece it together.

10. Summary of All Decisions

AREA	DECISION
experience / expectedSalary	Stored as numbers, not free text
appliedOn	Stored as a proper date/time value, not a plain text string
currentCTC, noticePeriod	Added as new fields
Candidate stages	The original 8, plus Offer Declined and On Hold, for 10 total
Application sources	The original 7, plus Company Website/Career Page, for 8 total
Resume storage	Filename plus a storage link, saved at upload time. PDF only. Original name plus a serial number. Old file deleted whenever it's replaced
Rejection reason	A dropdown with fixed options, applying to both the Rejected and Offer Declined stages
Interviews and feedback	A hybrid setup: full data lives in separate collections, with a small <code>nextInterview</code> summary also stored on each candidate's own record
Ownership and accountability	Both <code>assignedRecruiter</code> and <code>lastUpdatedBy</code> are kept, since they serve different purposes

Status: this schema is fully finalized. Nothing about the field layout remains undecided. The next step is implementation: updating the backend's field list for the `candidates` collection, and wiring the frontend up through the existing `HrDeskContext`, following the usage-limit-friendly practices described in section 9.